

Tema 9: Implementació de vectors

1. Estructura interna i Atributs

Un vector és un **array dinàmic** al *heap*. Per gestionar-lo, usem un **namespace** propi i tres atributs:

- **T* data_**: Punter al bloc de memòria on guardem els elements.
- **int size_**: Elements ocupats actualment.
- **int capacity_**: Memòria total reservada.

```
namespace pro2 {  
    template <typename T>  
    class Vector {  
        T* data_;  
        int size_, capacity_;  
        void reallocate_(int new_cap); // La "mudanza"  
    public:  
        using iterator = T*; // Iterador simple = Punter  
        // ... mètodes ...  
    };  
}
```

2. Regla de tres (Gestió de memòria)

Si una classe gestiona memòria dinàmica, necessita aquests 3 mètodes per evitar **segfaults** o **leaks**:

A. Constructor de còpia (Deep Copy)

Crea una còpia real en un nou bloc de memòria, no només copia el punter.

```
Vector(const Vector& v) {  
    data_ = new T[v.capacity_];  
    size_ = v.size_;  
    capacity_ = v.capacity_;  
    for (int i = 0; i < size_; ++i) data_[i] = v.data_[i];  
}
```

B. Operador d'assignació

Allibera la memòria vella abans de copiar la nova.

```
Vector& operator=(const Vector& v) {  
    if (this != &v) {  
        delete[] data_; // 1. Neteja  
        data_ = new T[v.capacity_]; // 2. Reserva  
        size_ = v.size_;  
        capacity_ = v.capacity_;  
        for (int i = 0; i < size_; ++i) data_[i] = v.data_[i]; // 3. Copia  
    }  
}
```

```
    return *this;
}
```

C. Destructor

L'únic que allibera la memòria definitivament.

```
~Vector() { delete[] data_; }
```

3. Operadors

Categoria	Definició de la funció (cpp)	Crida des del main
Accés	<code>T& operator[](int i)</code>	<code>v[i] = x;</code>
Comparació	<code>bool operator<(const V& v)</code>	<code>if (a < b)</code>
Assignació	<code>void operator+=(const V& v)</code>	<code>a += b;</code>
Aritmètics	<code>V operator+(const V& v)</code>	<code>c = a + b;</code>
Flux	<code>ostream& operator<<(ostream& o, const V& v)</code>	<code>cout << v;</code>

```
Stack<int> s1, s2;
s1 += s2;           // Sintaxi natural (al main)
s1.operator+=(s2);  // Equivalent a s1 += s2;
```

Quan tu escrius `s += s2` al main, el compilador de C++ busca si la classe té definida una funció que es digui literalment `operator+=`. Si la troba i accepta els arguments que li passes (en aquest cas un objecte de tipus `Stack`), fa la crida directa.

4. Accés i iteradors

L'accés és $\Theta(1)$ directe per aritmètica de punters. - **L'operador []**: Es duplica per permetre lectura en objectes `const`. - **Iteradors**: En un vector, un `iterator` és simplement un `T*`.

```
T& operator[](int i) { return data_[i]; }
const T& operator[](int i) const { return data_[i]; }
```

```
iterator begin() { return data_; }
iterator end() { return data_ + size_; }
```

5. El motor: `reallocate_` (La mudanza)

Mètode privat que canvia la capacitat del vector. Operació cara $\mathcal{O}(n)$. 1. Demana nou bloc. 2. Copia elements vells. 3. `delete[]` bloc vell. 4. Actualitza punter i capacitat.

```
void reallocate_(int new_cap) {
    T* new_data = new T[new_cap];           // 1. Nou bloc
    for (int i = 0; i < size_; ++i) {
        new_data[i] = data_[i];             // 2. Copia
    }
    delete[] data_;                          // 3. Neteja vell
    data_ = new_data;                        // 4. Actualitza
    capacity_ = new_cap;
}
```

6. Inserció i creixement

- **push_back**: Si està ple, **dobla** la capacitat.
- **Cost Amortitzat**: Tot i que redimensionar és $\mathcal{O}(n)$, només passa cada 2^k vegades. La mitjana és $\mathcal{O}(1)$.

```
void push_back(const T& x) {
    if (size_ == capacity_) {
        reallocate_(capacity_ == 0 ? 1 : 2 * capacity_);
    }
    data_[size_++] = x;
}

::vectorviz
```

7. Extracció i Thrashing

Per evitar oscil·lacions cares (doblar/reduir constantment): - **Estratègia**: Esperar a estar a **1/4** de capacitat per reduir-la a la **meitat**.

```
void pop_back() {
    --size_;
    if (size_ > 0 && size_ == capacity_ / 4) {
        reallocate_(capacity_ / 2);
    }
}
```

Resum de Complexitat

Operació	Complexitat	Nota
Accés [i]	$\Theta(1)$	Directe.
push_back	$\mathcal{O}(1)^*$	*Amortitzat. Pitjor cas $\mathcal{O}(n)$.
pop_back	$\mathcal{O}(1)^*$	*Amortitzat. Evitem el Thrashing.
insert/erase	$\mathcal{O}(n)$	Cal desplaçar tots els elements posteriors.
size/empty	$\Theta(1)$	Consulta d'atributs.

Extra: Operador de sortida

Molt útil per debugar la implementació:

```
template <typename T>
ostream& operator<<(ostream& os, const Vector<T>& v) {
    os << "[";
    for (int i = 0; i < v.size(); ++i) {
        os << v[i] << (i == v.size() - 1 ? "" : ",");
    }
    return os << "]";
}
```